■ API LOGGING FRAMEWORK

Production-Grade Full-Stack Platform for Centralized API Observability

Capture every request & response, monitor performance, and give teams complete visibility — without slowing it down. Built through 3 real implementations.

Laravel Backend API | React Axios | Dual DB PostgreSQL | Sanctum Auth | AWS EC2 + Nginx

■ Not a bolt-on library — fully engineered observability layer

■ Every outcome 200 • 201 • 400 • 401 • 403 • 404 • 422 • 500 captured via global middleware + terminate()

■ CORE PUFFS

- ✓ Every Request Logged
- ✓ Zero-Latency terminate()
- ✓ Global Middleware
- ✓ Dual DB Isolation
- ✓ Role-Based Access
- ✓ Advanced Filters
- ✓ AWS Production

■ Claymorphism Edition

■ TABLE OF CONTENTS — Claymorphism Edition

■ OVERVIEW — What problem does this solve?

■ THE PROBLEM — APIs invisible until break

- Scattered var_dump() calls
- Generic server logs
- Expensive APM tools

Who called? What sent? Why failed?

■ THE SOLUTION — Full observability

Captures, stores, filters logs for EVERY request & response — success/failure.

✓ Zero impact response time
✓ Fully engineered, not bolt-on

■ TWO ROLES. ONE PLATFORM.

■ Professor → Create/update/delete notes [WRITE+READ]

■■■ Student → Read-only notes [READ ONLY]

■ Differentiator: Not a logging library. It's a production observability layer with bubbly clay UI!

■ TECH STACK — Every tech chosen deliberately

■ Backend

Laravel PHP

REST APIs routing

■■ Frontend

React Axios

SPA client HTTP UI

■ Auth

Sanctum

Revocable tokens

■ Primary DB

PostgreSQL pgsql

Students notes tokens

■ Logging DB

log_pgsql

Isolated api_logs

■■ Cloud

AWS EC2

Prod hosting

■■ SYSTEM ARCHITECTURE — Request Flow

CLIENT BROWSER	React → Axios
GLOBAL MIDDLEWARE	handle() start timer
SANCTUM AUTH	Validate token
CONTROLLER	Business logic CRUD
MAIN DB	pgsql students notes
RESPONSE ■	Zero wait
TERMINATE	terminate() log_pgsql

■ WHY IT MATTERS — Clay View

Global Guarantee

In Kernel.php $middleware, not api group.
Runs on EVERY incl. 401/403/404.

Zero Latency

terminate() after response sent.

DB write behind scenes.

Captured Fields

- user role method endpoint headers
- body req/res IP UA status time

■ All: 200 201 400 401 403 404 422 500

■ Global Middleware

Groups only after router → 401/403/404 invisible. Global runs BEFORE routing → 100% visibility.

■ terminate() vs handle()

Inline = client waits DB. terminate() AFTER response → zero latency, same lifecycle.

■ Why Not Queue?

Queue needs worker + jobs table → overkill. Terminable achieves same benefit simpler.

■ Why Two DBs?

Log writes high volume. Isolation prevents lock contention on biz tables.

- ■ Terminate Zero-Latency OK
- ■ Dual DB OK
- ■ Filter API OK
- ■ AWS EC2 Nginx OK
- ■ Backup OK
- ■ Postman Verified OK
- ■ Sanctum Auth OK
- ■ RBAC OK
- ■ Email Alerts OK
- ■ Student CRUD OK
- ■ Notes OK
- ■ Global Logging OK

■ THANK YOU

API Logging Framework

Production-Grade • Full-Stack • Observability Platform

Laravel • React • PostgreSQL Dual DB • Sanctum • AWS EC2 • Nginx

Built to capture every request & response without slowing it down
Global middleware • terminate() zero-latency • dual-database isolation

■ Soft • Bubbly • Clay • 3D • Pastel • Chunky Borders • Thick Shadows

■ Ready for Production • ■ Complete Visibility • ■ Secure & Scalable